

Notion 智慧排程代理系統

摘要

本報告提出一套 Notion 智慧排程代理系統，目標是讓使用者能以自然語言提出排程需求，並由大型語言模型（Large Language Model, LLM）作為系統控制器，自動完成時間解析、行程查詢、衝突檢查與 Notion 行程建立或更新。本系統重點不在模型訓練，而在於 AI Harness 的設計：如何讓 LLM 透過 function calling、tools、memory 與 workflow orchestration 完成多步驟任務。

從決策系統角度來看，排程任務可被視為一個序列決策問題。Agent 必須根據使用者意圖、目前時間、既有行程與工具回傳結果，選擇下一步應呼叫的工具。透過明確的工具設計與流程控制，本系統能降低手動排程的認知負擔，並避免在時間未確認或行程衝突尚未檢查前直接寫入 Notion。

1. 問題定義與應用背景

在日常工作與學習環境中，使用者經常需要安排會議、課程、專案任務或個人行程。傳統排程工具雖然能儲存結構化資料，但通常要求使用者手動輸入明確的日期、時間、標題與分類。當使用者提出較模糊的需求，例如「下週找個下午開會」、「幫我把明天的討論改晚一點」或「找一個不衝突的時間」時，傳統工具缺乏自然語言理解與自動決策能力。

因此，本系統設計一個 Notion 智慧排程代理系統，讓使用者可透過 Discord 等通訊介面輸入自然語言請求。系統接收訊息後，會由 LLM 判斷使用者意圖，並透過工具調用取得目前時間、解析時間範圍、查詢 Notion Database 中既有行程，最後在確認無衝突或取得使用者同意後建立或更新行程。

此應用情境符合 AI Harness 的核心精神：LLM 不只是生成回覆，而是作為 system controller，負責規劃任務流程、選擇工具、整合工具結果，並控制整體 workflow。

2. AI Harness 系統設計

本系統由四個主要部分組成：使用者介面、LLM Agent、Tool Layer 與 Memory。

2.1 使用者介面

使用者透過 Discord Bot 輸入排程請求。例如：

「幫我安排明天下午三點到四點和 John 開會。」

Discord Bot 會將訊息傳送給後端的 Agent Controller，並在任務完成或需要確認時將結果回覆給使用者。

2.2 LLM Agent Controller

LLM Agent Controller 是系統核心，負責理解使用者意圖並決定下一步行動。它不直接修改 Notion，而是透過 function calling 呼叫外部工具。Agent 的主要任務包括：

- 1. 判斷使用者是要新增、查詢或修改行程。
- 2. 判斷是否需要取得目前時間。
- 3. 呼叫時間解析工具。
- 4. 查詢 Notion 行程是否衝突。
- 5. 根據工具結果決定是否建立、更新或詢問使用者確認。

2.3 Tool Layer

Tool Layer 提供標準化 API，讓 LLM 可以在受控範圍內操作外部系統。這能避免 LLM 直接產生不可靠或不可執行的操作，也讓每一步行為能被記錄與評估。

2.4 Memory 設計

本系統包含兩種記憶：

短期記憶：保存當前對話上下文。例如，當系統詢問「是否改到四點？」而使用者回答「好」時，Agent 必須知道這個「好」是指原本那場會議。

長期記憶：可用於記錄使用者偏好，例如偏好下午開會、不希望午餐時間安排會議等。此部分可作為未來擴充功能，但 MVP 階段可先以短期記憶為主。

3. Function Calling 與 Tools 設計

本系統透過 function calling 讓 LLM 在受控範圍內調用外部工具，而非直接產生不可驗證的操作。LLM Agent 會根據使用者意圖與目前任務狀態，選擇適當的 API 工具，例如取得目前時間、解析自然語言時間、查詢 Notion 行程，以及建立或更新任務。此設計可使系統具備可追蹤性與可評估性，並降低 LLM 誤判後直接寫入資料庫的風險。

本系統設計五個核心工具，如表 1 所示。

表 1：核心工具設計

工具名稱	功能說明	使用時機
get_current_time()	取得目前時間與時區，作為解析相對時間的基準。	使用者輸入「明天」、「下週」等相對時間時。
parse_time_range(text, reference_time)	將自然語言時間轉換為標準起訖時間，並回傳解析信心分數。	需要將模糊時間轉為可查詢或寫入的格式時。
query_notion_schedule(start_time, end_time)	查詢指定時間範圍內是否已有行程，避免時間衝突。	建立或更新行程前必須執行。

工具名稱	功能說明	使用時機
<code>create_notion_task(task_info)</code>	在 Notion Database 中建立新行程。	時間已明確且確認無衝突時。
<code>update_notion_task(task_id, updated_info)</code>	修改既有 Notion 行程，例如改期或更新內容。	使用者要求修改既有行程時。

在工具使用邏輯上，系統採取「先觀測、再判斷、後寫入」的原則。也就是說，Agent 不可在未解析時間或未查詢衝突前直接呼叫 `create_notion_task()` 或 `update_notion_task()`。例如，當使用者輸入「幫我安排明天下午三點到四點和 John 開會」時，Agent 會先取得目前時間，再解析時間範圍，接著查詢 Notion 是否已有衝突行程。只有在查詢結果顯示無衝突時，系統才會建立新行程。

若 `parse_time_range()` 回傳的信心分數過低，Agent 應要求使用者補充資訊；若 `query_notion_schedule()` 回傳衝突事件，Agent 則必須暫停寫入，向使用者說明衝突並提出替代時段。此 function calling 設計使 LLM 的動作空間受到明確限制，也讓整體 workflow 更安全且具可解釋性。

4. Agent Workflow 與 AI Orchestration

本系統採用意圖驅動的多步驟 workflow。Agent 不依賴固定單一路徑，而是根據使用者意圖與工具結果動態決定下一步。

4.1 新增行程流程

以「幫我安排明天下午三點到四點和 John 開會」為例，流程如下：

1. Agent 判斷使用者意圖為新增行程。
2. 呼叫 `get_current_time()` 取得目前時間。
3. 呼叫 `parse_time_range()` 解析「明天下午三點到四點」。
4. 若解析信心分數足夠，呼叫 `query_notion_schedule()` 查詢是否衝突。
5. 若沒有衝突，呼叫 `create_notion_task()` 建立行程。
6. 回覆使用者建立結果。

4.2 衝突處理流程

若 `query_notion_schedule()` 回傳已有行程，Agent 不可直接建立新行程，而應回覆使用者：

「該時段與 Project Meeting 衝突，是否要改到下午四點到五點？」

只有在使用者確認後，Agent 才能重新查詢替代時段並建立行程。此設計確保 human-in-the-loop，避免系統擅自覆寫或忽略既有安排。

4.3 修改行程流程

若使用者輸入「把明天和 John 的會議改到四點」，Agent 應先判斷這是修改行程，而非新增行程。流程如下：

1. 查詢既有行程，找出 John 相關會議。
2. 解析新時間。
3. 查詢新時間是否衝突。
4. 若無衝突，呼叫 `update_notion_task()`。
5. 回覆修改結果。

4.4 決策與流程控制

從 AI orchestration 的角度來看，Agent 的每一步行動都受到條件限制：

- 未取得目前時間前，不應解析相對時間。
- 未完成時間解析前，不應查詢行程。
- 未查詢衝突前，不應建立或更新行程。
- 若存在衝突，必須先取得使用者確認。
- 只有在條件滿足時，才能呼叫寫入型工具。

此流程控制能讓 LLM 的行動空間受到約束，降低錯誤工具調用與不合理排程的風險。

5. 決策觀點：MDP 簡化分析

本系統可簡化為一個序列決策問題。Agent 在每一步會根據目前狀態選擇下一個工具。

State Space 包含使用者輸入、對話上下文、目前時間、時間解析結果、Notion 查詢結果與工具錯誤訊息。

Action Space 則是 Agent 可呼叫的工具，例如 `get_current_time()`、`parse_time_range()`、`query_notion_schedule()`、`create_notion_task()` 與 `update_notion_task()`。

Reward Signal 可用於評估決策品質。成功建立或更新無衝突行程可視為正向結果；若 Agent 未查詢衝突就寫入、誤用 `create` 代替 `update`、或反覆詢問已知資訊，則視為負向結果。

本作業不需要實際訓練 DRL 模型，但此觀點有助於分析 AI Harness 的設計品質。換言之，`system prompt`、`function schema` 與 `workflow rules` 可被視為人工設計的策略，使 Agent 能在有限工具集合中做出合理決策。

6. Evaluation 方法

本系統可從功能正確性、工具選擇與流程安全性三個面向進行評估。

6.1 時間解析正確率

測試多種自然語言時間輸入，例如「明天下午」、「下週三早上」、「兩小時後」，檢查 `parse_time_range()` 是否能正確轉換為標準時間格式。

6.2 衝突偵測攔截率

預先在 Notion Database 中安排重疊行程，再要求 Agent 建立同時段行程。若系統能偵測衝突並暫停寫入，代表衝突處理有效。

6.3 工具選擇準確率

評估 Agent 是否根據意圖選擇正確工具。例如，新增行程應使用 `create_notion_task()`，修改行程應使用 `update_notion_task()`。若使用者要求改期，但 Agent 建立了一個新行程，則代表工具選擇錯誤。

6.4 Workflow 完成率

測量 Agent 是否能順利完成完整流程，包括時間解析、衝突查詢、使用者確認與最終寫入。此指標可反映整體 orchestration 是否穩定。

6.5 錯誤恢復率

模擬 Notion API 回傳錯誤，例如權限不足、欄位格式錯誤或找不到 `task_id`，觀察 Agent 是否能正確回報問題或重新嘗試合理修正。

6.6 使用者確認遵守率

在衝突情境下，檢查 Agent 是否確實等待使用者確認後才建立或更新行程。此指標可衡量系統是否遵守安全流程。

7. 結論

本報告提出之 Notion 智慧排程代理系統，展示了 AI Harness 在實際應用中的設計方式。系統以 LLM 作為控制器，結合 tools、memory 與 workflow orchestration，使自然語言排程需求能被轉換為具體且安全的 Notion 操作。

本系統設計包含問題定義、AI system architecture、function calling、至少三個工具、agent workflow、evaluation 方法與 AI orchestration。透過時間解析、衝突查詢與條件式寫入，本系統能避免傳統手動排程的低效率，也能降低 LLM 直接操作外部系統時可能產生的錯誤。

整體而言，此設計符合 HW4 對 AI Harness Systems Design and Analysis 的要求，並呈現出 LLM 作為 system controller 時，如何透過工具調用與流程控制完成多步驟任務。